

CAPÍTULO 4

ALGORITMOS Y ESTRUCTURAS DE CONTROL REPETITIVAS EN LENGUAJE C/C++



Margarita Zambrano, César Villacís, David Alvarado, Luis García, Kevin Topón,
Margarita Villacís, Sofía Bayona, Luis Pastor

Universidad de las Fuerzas Armadas – ESPE
Quito, Ecuador

Universidad Politécnica Salesiana – UPS
Quito, Ecuador

Universidad Rey Juan Carlos – URJC
Madrid, España

Contenido

- **La sentencia do-while**

- Problema 4.5: (Aplicación de la sentencia do-while como un bucle ascendente)
- Problema 4.6: (Aplicación de la sentencia do-while como un bucle descendente)
- Caso de Estudio : Análisis de Números Palíndromos

La sentencia do-while

La sentencia do-while() crea un bucle que se repite hasta que la expresión lógica o condición del bucle se vuelva falsa. El cuerpo del bucle do-while() se ejecutará una o varias veces, pero al menos una vez, dependiendo de la condición del bucle (Hanly, H., Koffman, E., 2013). El formato de esta sentencia tiene la siguiente sintaxis.

Sintaxis 1:

```
do
    sentencia;
while(Condición_Bucle);
```

Condición_Bucle :Expresión lógica o booleana que determina si la sentencia o acción se ha de ejecutar de manera repetitiva.

sentencia: La sentencia o acción se ejecuta si la expresión lógica es verdadera dentro del bucle.

Nota: En esta sintaxis no es necesario utilizar llaves ya que se tiene una sola sentencia o instrucción.

La sentencia do-while

Condición_Bucle :Expresión lógica o booleana que determina si las sentencias o acciones se han de ejecutar de manera repetitiva.

sentencias: Las acciones o sentencias se ejecutan si la expresión lógica es verdadera dentro del bucle.

Sintaxis 2:

```
do
{
    sentencia1;
    sentencia2;
    ...
    sentencian;
} while(Condición_Bucle);
```

La sentencia do-while

- La sentencia do-while() crea un bucle que se repite hasta que la expresión lógica o condición del bucle se vuelva falsa. El cuerpo del bucle do-while() se ejecutará una o varias veces, pero al menos una vez, dependiendo de la condición del bucle (Hanly, H., Koffman, E., 2013). El formato de esta sentencia tiene la siguiente sintaxis

Sintaxis 1:

```
do
    sentencia;
while(Condición_Bucle);
```

Imagen 4.1 Representación del lenguaje de programación C.

Fuente: MSMK University, 2025.

Contenido

- La sentencia do-while
- **Problema 4.5: (Aplicación de la sentencia do-while como un bucle ascendente)**
- Problema 4.6: (Aplicación de la sentencia do-while como un bucle descendente)
- Caso de Estudio : Análisis de Números Palíndromos

Problema 4.5: (Aplicación de la sentencia do-while como un bucle ascendente)

- Problema 4.5: Escribir un programa que permita imprimir de manera ascendente los números del 1 al 10 utilizando un bucle do-while().

Nº	Algoritmo en Pseudocódigo	Algoritmo en Lenguaje C/C++
1	entero : i	<code>int i;</code>
2	Inicializar el valor del contador en uno.	<code>i = 1;</code>
3	Hacer:	<code>do {</code>
4	Imprimir el valor del contador.	<code>cout << i << endl;</code>
5	Incrementar el valor del contador en uno.	<code>i++;</code>
6	Mientras la condición sea verdadera.	<code>} while (i <= 10);</code>

Problema 4.5: (Aplicación de la sentencia do-while como un bucle ascendente)

```
// Librerías.
#include <iostream>
#include <cstdlib>
#include <cmath>

using namespace std;

// Función principal.
int main()
{
    // Declaración de variables.
    int i; // Salida: contador del bucle.

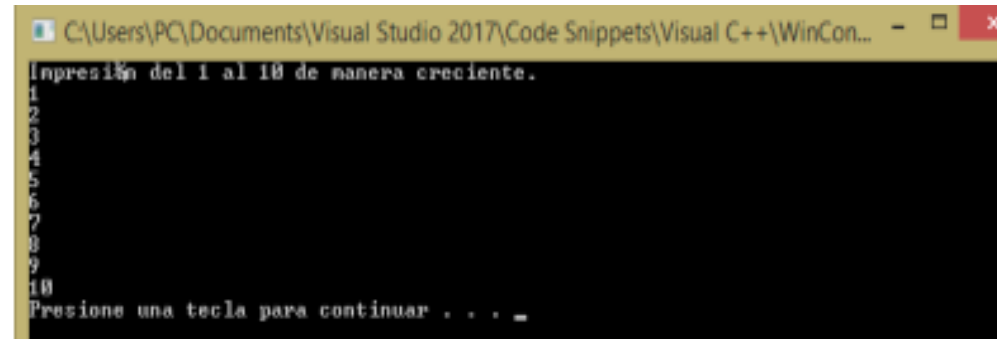
    // Impresión de un mensaje de información.
    cout << "Impresión del 1 al 10 de manera creciente." << endl;

    // Inicializar el valor del contador en uno.
    i = 1;
    // Hacer:
    do {
        cout << i << endl;
        // Incrementar el valor del contador en uno.
        i++;
    } while (i <= 10);

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Problema 4.5: (Aplicación de la sentencia do-while como un bucle ascendente)

En la salida de la Ejecución 4.5 se puede ver la impresión del 1 al 10 de manera creciente.



```
C:\Users\PC\Documents\Visual Studio 2017\Code Snippets\Visual C++\WinCon...
Impresión del 1 al 10 de manera creciente.
1
2
3
4
5
6
7
8
9
10
Presione una tecla para continuar . . . _
```

Ejecución del Programa 4.5 Ejecución del Programa Fuente:
Zambrano M., et al., 2025.

Problema 4.5: (Aplicación de la sentencia do-while como un bucle ascendente)

- En la Tabla 4.5 se puede ver la prueba de escritorio del programa, donde se puede ver que en cada iteración del bucle el contador se incrementa en uno, mientras la condición del bucle sea verdadera.

i	i <= 10	cout << i << endl;
1		1 INTRO
2	2 <= 10 (V)	2 INTRO
3	3 <= 10 (V)	3 INTRO
4	4 <= 10 (V)	4 INTRO
5	5 <= 10 (V)	5 INTRO
6	6 <= 10 (V)	6 INTRO
7	7 <= 10 (V)	7 INTRO
8	8 <= 10 (V)	8 INTRO
9	9 <= 10 (V)	9 INTRO
10	10 <= 10 (V)	10 INTRO
11	11 <= 10 (F)	

Contenido

- La sentencia do-while
- Problema 4.5: (Aplicación de la sentencia do-while como un bucle ascendente)
- Problema 4.6: (Aplicación de la sentencia do-while como un bucle descendente)
- **Caso de Estudio : Análisis de Números Palíndromos**

Caso de Estudio : Análisis de Números Palíndromos

- A) Problema

Escribir un programa que permita analizar si un número es palíndromo o no lo es. Un número palíndromo es aquel número que invertido o dado la vuelta resulta ser el mismo número. Así, por ejemplo, los números 1234321, 55555, 333, son números palíndromos. Analizar además el número de cifras que tiene el número, visualizar el número original al revés y desplegar un mensaje donde se mencione si el número ingresado por teclado es o no palíndromo.

Caso de Estudio : Análisis de Números Palíndromos

- B) Análisis

Claramente se puede ver que la entrada del problema es un número entero. Hay dos salidas requeridas que son el número de cifras del número y el número original invertido. Se requieren seis datos auxiliares que son: un contador de cifras para el número, el dividendo de la división, el cociente de la división, el residuo de la división, una sumatoria para acumular valores y un contador para las potencias de 10. En la solución computacional de análisis de un número palíndromo se plantea primeramente un bucle que cuenta el número de cifras del número entero leído desde el teclado, como se puede ver en la Figura 4.4.1.

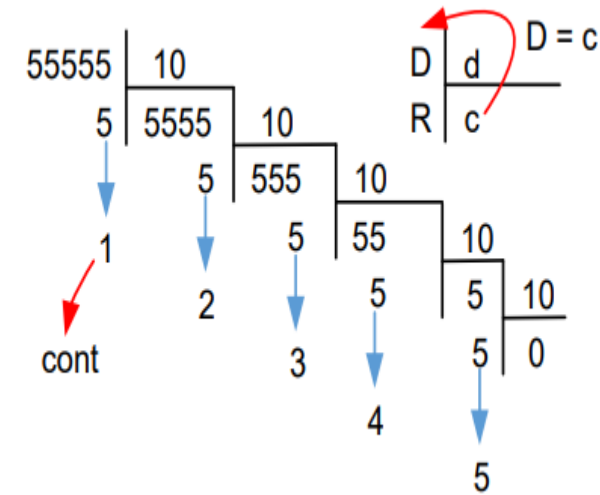


Figura 4.4.1 Proceso para contar el número de cifras de un número entero. Fuente: Zambrano M., et al., 2025.

Caso de Estudio : Análisis de Números Palíndromos

Luego, hay un bucle que controla las divisiones sucesivas, que permiten obtener los restos de las divisiones que se acumulan en una sumatoria que acumula los resultados parciales del producto del residuo por 10 elevado a una potencia de 'n' y un contador que controla los valores de la potencia de 'n'. Antes de ingresar a este bucle, el contador se inicializa con el valor del número de cifras menos el valor de uno. De manera sucesiva, el resultado del cociente de una división pasa a ser el dividendo de la siguiente división, asignando a la variable dividendo el valor del cociente en cada iteración del bucle. Finalmente, el valor de la sumatoria obtenida viene a ser el valor del número invertido calculado, como se puede ver en la Figura 4.4.2. El requerimiento de los datos y las fórmulas requeridas se muestran a continuación.

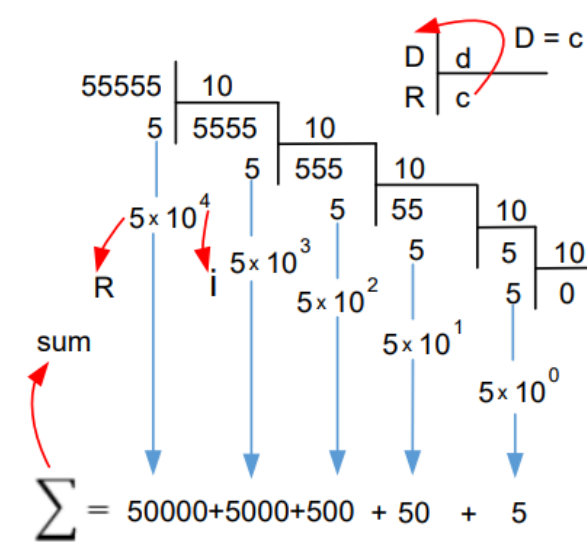


Figura 4.4.2 Proceso para invertir un número entero. Fuente: Zambrano M., et al., 2025.

Caso de Estudio : Análisis de Números Palíndromos

B.1) Requerimiento de los Datos

Entradas del Problema

num /* número entero original */

Salidas del Problema

numeroCifras /* número de cifras decimales del entero */
numInvertido /* número entero invertido */
Mensaje /* que indica que el número ingresado es palíndromo o no es
palíndromo */

Auxiliares del Problema

cont /* contador de cifras decimales de un entero */
D /* Dividendo de la división */
c /* cociente de la división */
R /* Residuo de la división */
sum /* sumatoria */
i /* contador */

Fórmulas Relevantes

$c = D \div 10$ (1) /* Fórmula que calcula el cociente de una división */
 $R = D \text{ MOD } 10$ (2) /* Fórmula que calcula el residuo de una división */
 $sum = sum + R \cdot 10^i$ (3) /* Fórmula que acumula el valor de una sumatoria más el
producto entre el residuo por 10 elevado a la potencia de 'i' */

Caso de Estudio : Análisis de Números Palíndromos

B.2) Diagrama de Entrada-Salida

En la Figura 4.4.3 se muestra el diagrama de Entrada-Salida del programa donde se identifican y se diagraman las entradas, salidas y auxiliares del problema como son:

a) Entradas: la variable 'num';

b) Salidas: la variable 'numeroCifras'; la variable 'numInvertido'; un mensaje de información que indica que el número ingresado es palíndromo o no es palíndromo;

c) Auxiliares: la variable 'cont'; la variable 'D'; la variable 'c'; la variable 'R'; la variable 'sum'; la variable 'i'.

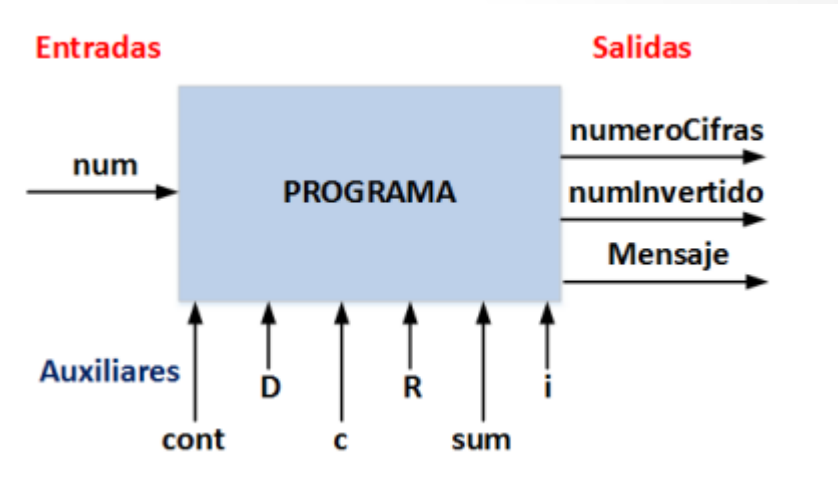


Figura 4.4.3 Diagrama de entrada –salida .
Fuente: Zambrano M., et al., 2025.

Caso de Estudio : Análisis de Números Palíndromos

C) Diseño

1. Imprimir el mensaje de información: Analizar si un número es palíndromo.
2. Leer el valor de un número entero, utilizando la variable 'num'.
3. Asignar a la variable 'D' (Dividendo) el valor del número entero (num).
4. Inicializar el contador 'cont' con el valor de cero.
5. Hacer:
 - 5.1. Calcular el valor del cociente (c) entre el Dividendo (D) y el valor de 10.
 - 5.2. Asignar a la variable Dividendo (D) el valor del cociente (c).
 - 5.3. Incrementar el valor del contador 'cont' en uno.Mientras el cociente de la división sea mayor que cero.
6. Asignar a la variable 'numeroCifras' el valor del contador 'cont' que corresponde al valor del número de cifras del número entero ingresado.
7. Imprimir el número de cifras del número entero ingresado por teclado.
8. Asignar a la variable 'D' (Dividendo) el valor del número entero (num).
9. Inicializar la variable 'sum' (sumatoria) con el valor de cero.
10. Inicializar el contador 'i' con el valor del número de cifras (numeroCifras) menos uno.
11. Hacer:
 - 11.1. Calcular el valor del cociente (c) entre el Dividendo (D) y el valor de 10.
 - 11.2. Calcular el valor del Residuo (R) entre el Dividendo (D) y el valor de 10.
 - 11.3. Asignar a la sumatoria (sum) el valor que tiene acumulado más el producto entre el Residuo (R) con el valor de 10 elevado a la potencia de 'i'.
 - 11.4. Asignar a la variable 'D' (Dividendo) el valor del cociente.
 - 11.5. Decrementar el valor del contador 'i' en uno.Mientras el cociente de la división sea mayor que cero.
12. Asignar a la variable 'numInvertido' el valor de la sumatoria 'sum' que corresponde al valor del número entero invertido.
13. Imprimir el número entero invertido.
14. Si el valor de la variable 'num' que tiene el valor del número entero original es igual al valor de la variable 'numInvertido' que tiene el valor del número entero invertido.
 - 14.1. Imprimir el mensaje: "El número ingresado es palíndromo".
15. Caso contrario, el valor de la variable 'num' es diferente del valor de la variable 'numInvertido'.
 - 15.1. Imprimir el mensaje: "El número ingresado no es palíndromo".

Caso de Estudio : Análisis de Números Palíndromos

D) Implementación

Para implementar la solución, se debe escribir el algoritmo como un programa de C/C++ que contenga toda la información necesaria para completar la traducción a lenguaje máquina. En la Tabla 4.4.1 se muestra el código del programa en C/C++.

```
// Librerías.
#include <iostream>
#include <cstdlib>
#include <cmath>

using namespace std;

// Función principal.
int main()
{
    // Declaración de variables.
    long num;           // Entrada: Número original.
    long cont;          // Auxiliar: Contador de cifras decimales.
    long numeroCifras;  // Salida: Número de cifras decimales.
    long numInvertido;  // Salida: Número invertido.
    long D;             // Auxiliar: Dividendo.
    long c;             // Auxiliar: cociente.
    long R;             // Auxiliar: Residuo.
```

```

long sum;           // Auxiliar: Sumatoria.
long i;             // Auxiliar: contador.

// Imprimir un mensaje de información.
cout << "\tAnalizar si un número es Palíndromo." << endl << endl;
// Leer el número entero.
cout << "Ingrese un número: "; cin >> num;

// Asignar a la variable Dividendo el valor del número entero.
D = num;
// Inicializar el contador 'cont' con el valor de cero.
cont = 0;
// Hacer:
do {
    // Calcular el valor del cociente entre el Dividendo y el valor de 10.
    c = D / 10;
    // Calcular el valor del Residuo entre el Dividendo y el valor de 10.
    D = c;
    // Incrementar el valor del contador 'cont' en uno.
    cont++;
    // Mientras el cociente de la división sea mayor que cero.
} while (c > 0);
// Asignar a la variable 'numeroCifras' el valor del contador 'cont' que
// corresponde al valor del número de cifras del número entero ingresado.
numeroCifras = cont;
// Imprimir el número de cifras del número entero ingresado por teclado.
cout << endl << "El número tiene " << numeroCifras << " cifras." << endl;

// Invertir el número.
// Asignar a la variable Dividendo el valor del número entero.
D = num;
// Inicializar la variable sumatoria con el valor de cero.
sum = 0;
// Inicializar el contador 'i' con el valor del número de cifras calculado.
i = numeroCifras - 1;
// Hacer:
do {
    // Calcular el valor del cociente entre el Dividendo y el valor de 10.
    c = D / 10;
    // Calcular el valor del Residuo entre el Dividendo y el valor de 10.
    R = D % 10;
    // Asignar a la sumatoria el valor que tiene acumulado más el producto
    // entre el Residuo con el valor de 10 elevado a la potencia de 'i'.
    sum = sum + R * pow(10, i);
    // Asignar a la variable Dividendo el valor del cociente.
    D = c;
    // Decrementar el valor del contador 'i' en uno.
    i--;
    // Mientras el cociente de la división sea mayor que cero.
} while (c > 0);
// Asignar a la variable 'numInvertido' el valor de la sumatoria 'sum'
// que corresponde al valor del número entero invertido.
numInvertido = sum;
// Imprimir el número entero invertido.
cout << "El número invertido es: " << numInvertido << endl;

// Si el valor de la variable 'num' que tiene el valor del número entero
// original es igual al valor de la variable 'numInvertido' que tiene
// el valor del número entero invertido.
if (num == numInvertido)
{

```

Caso de Estudio : Análisis de Números Palíndromos

D) Implementación

Para implementar la solución, se debe escribir el algoritmo como un programa de C/C++ que contenga toda la información necesaria para completar la traducción a lenguaje máquina. En la Tabla 4.4.1 se muestra el código del programa en C/C++.

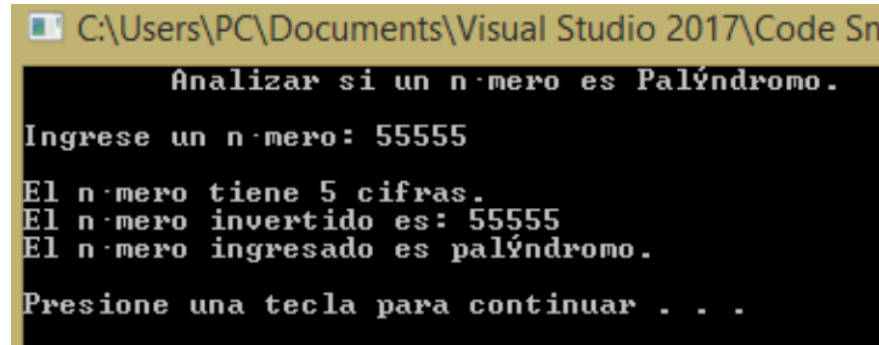
```
        cout << "El número ingresado es palíndromo." << endl << endl;
    }
    // Caso contrario, el valor de la variable 'num' es diferente del valor
    // de la variable 'numInvertido'.
    else
    {
        //Imprimir el mensaje: El número ingresado no es palíndromo.
        cout << "El número ingresado no es palíndromo." << endl << endl;
    }

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Caso de Estudio : Análisis de Números Palíndromos

E) Pruebas

En la Tabla 4.4.2, se muestra un ejemplo de la salida del programa, donde se puede ver que los resultados del programa tienen sentido, comparando estos resultados con los datos devueltos por una calculadora. En la salida de la Tabla 4.4.2 se puede ver que, al ingresar el número 55555, el programa indica que este número tiene 5 cifras decimales. Luego, se obtiene el número invertido que en este caso resulta ser el mismo número, y por lo tanto el número ingresado es palíndromo.



```
C:\Users\PC\Documents\Visual Studio 2017\Code Sn
Analizar si un n-mero es Palíndromo.
Ingrese un n-mero: 55555
El n-mero tiene 5 cifras.
El n-mero invertido es: 55555
El n-mero ingresado es palíndromo.
Presione una tecla para continuar . . .
```

Ejecución del Programa 2.2.2 Ejecución del Programa Caso de Estudio. Fuente: Zambrano M., et al., 2025.

Caso de Estudio : Análisis de Números Palíndromos

En la Tabla 4.4.3, se muestra la prueba de escritorio del primer bucle do-while() del programa que cuenta el número de cifras que tiene el número ingresado por el teclado que en este caso es el número 55555 cuyo número de cifras es 5.

num	D	cont	c	c > 0
55555	55555	0		
	5555	1	5555	5555 > 0 (V)
	555	2	555	555 > 0 (V)
	55	3	55	55 > 0 (V)
	5	4	5	5 > 0 (V)
	0	5	0	0 > 0 (F)

Caso de Estudio : Análisis de Números Palíndromos

En la Tabla 4.4.4, se muestra la prueba de escritorio del segundo bucle do-while() del programa que invierte el número original que es el 55555 y se obtiene nuevamente el mismo número 55555. En cada división sucesiva el cociente de una división pasa a ser el dividendo de la siguiente división. También, se puede comprobar que el resultado de la sumatoria es el número invertido calculado mediante la suma de cada residuo multiplicado por una potencia de diez comenzando con la potencia igual al número de cifras menos uno hasta llegar al valor de cero. Finalmente, la ejecución de la sentencia do-while() termina cuando la condición del bucle se hace falsa. En este caso el número 55555 resulta ser palíndromo.

num	D	sum	i	c	R	c > 0
55555	55555	0	4			
	5555	$0+5 \times 10^4=50000$	3	5555	5	$5555 > 0$ (V)
	555	$50000+5 \times 10^3=55000$	2	555	5	$555 > 0$ (V)
	55	$55000+5 \times 10^2=55500$	1	55	5	$55 > 0$ (V)
	5	$55500+5 \times 10^1=55550$	0	5	5	$5 > 0$ (V)
	0	$55550+5 \times 10^0=55555$	-1	0		$0 > 0$ (F)

Caso de Estudio : Análisis de Números Palíndromos

En la Tabla 4.4.4, se muestra la prueba de escritorio del segundo bucle do-while() del programa que invierte el número original que es el 55555 y se obtiene nuevamente el mismo número 55555. En cada división sucesiva el cociente de una división pasa a ser el dividendo de la siguiente división. También, se puede comprobar que el resultado de la sumatoria es el número invertido calculado mediante la suma de cada residuo multiplicado por una potencia de diez comenzando con la potencia igual al número de cifras menos uno hasta llegar al valor de cero. Finalmente, la ejecución de la sentencia do-while() termina cuando la condición del bucle se hace falsa. En este caso el número 55555 resulta ser palíndromo.

```
Analizar si un n-mero es Palíndromo.  
Ingrese un n-mero: 1234  
El n-mero tiene 4 cifras.  
El n-mero invertido es: 4321  
El n-mero ingresado no es palíndromo.  
Presione una tecla para continuar . . .
```

Caso de Estudio : Análisis de Números Palíndromos

En la Tabla 4.4.6, se muestra la prueba de escritorio del primer bucle do-while() del programa que cuenta el número de cifras que tiene el número ingresado por el teclado que en este caso es el número 1234 cuyo número de cifras es 4.

num	D	cont	c	c > 0
1234	1234	0		
	123	1	123	123 > 0 (V)
	12	2	12	12 > 0 (V)
	1	3	1	1 > 0 (V)
	0	4	0	0 > 0 (F)

Caso de Estudio : Análisis de Números Palíndromos

En la Tabla 4.4.7, se muestra la prueba de escritorio del segundo bucle do-while() del programa que invierte el número original que es el número 1234 y se obtiene el número invertido 4321. En cada división sucesiva el cociente de una división pasa a ser el dividendo de la siguiente división. También, se puede comprobar que el resultado de la sumatoria es el número invertido calculado mediante la suma de cada residuo multiplicado por una potencia de diez comenzando con la potencia igual al número de cifras menos uno hasta llegar al valor de cero. Finalmente, la ejecución de la sentencia do-while() termina cuando la condición del bucle se hace falsa. En este caso el número 1234 resulta no ser palíndromo.

num	D	sum	i	c	R	c > 0
1234	1234	0	3			
	123	$0+4 \times 10^3=4000$	2	123	4	$123 > 0$ (V)
	12	$4000+3 \times 10^2=4300$	1	12	3	$12 > 0$ (V)
	1	$4300+2 \times 10^1=4320$	0	1	2	$1 > 0$ (V)
	0	$4320+1 \times 10^0=4321$	-1	0	1	$0 > 0$ (F)

Pensamiento

Científico de la Computación

- “Todo el mundo debería saber programar”

Steve Jobs
1955-2011

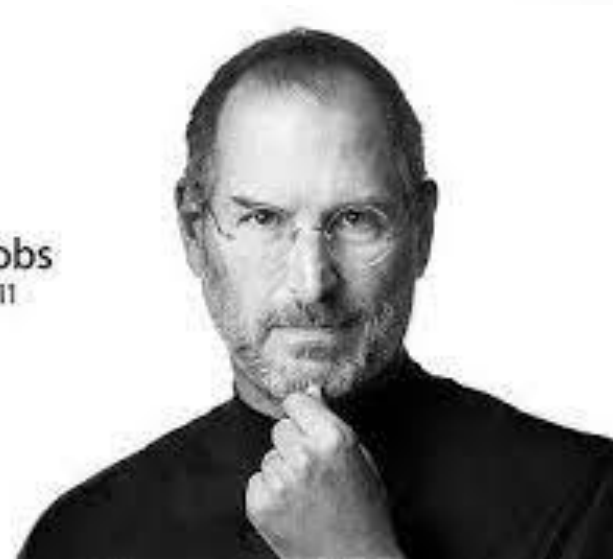


Figura 32. Universidad de las Fuerzas Armadas - ESPE.
Fuente: <https://economipedia.com/definiciones/steve-jobs.html>

¿Preguntas?

!Gracias!